The Situation

Imagine you built a website where customers can view their investment portfolios.

One day, customers start saying:

"The page keeps loading for 10 seconds or sometimes doesn't load at all."

This is a serious problem because customers need this information quickly.

Your job is to find **why** it's happening.

Step 1: Don't Guess

A beginner might say:

"The website is slow. Let's buy a bigger server."

But that's just guessing.

Instead, you investigate.

You ask:

- Is the frontend (Angular) slow?
- Is the backend (.NET API) slow?
- Is the database (SQL Server) slow?

You're trying to answer:

Where exactly is the delay?

Step 2: Follow One Request

Suppose a customer clicks **"View Portfolio."**

The request travels like this:

User

↓

Angular UI

↓

ASP.NET Web API

↓

SQL Server Database

↓

API

↓

Angular

↓

User sees data

You measure how long each step takes.

Example:

Step	Time
Angular → API	50 ms
API processing	100 ms
SQL Query	9,500 ms

Now it's obvious.

The database is taking almost all the time.

Step 3: Investigate the Database

Now you ask:

Why is the database so slow?

You open something called an **Execution Plan**.

What is an Execution Plan?

Think of Google Maps.

If you're driving from New York to Boston, Google Maps shows the best route.

SQL Server does the same thing.

When you ask for data, SQL Server decides:

"What's the fastest way to find this information?"

That decision is called the **Execution Plan**.

Step 4: Find the Problem

Suppose the database has:

10 million portfolio records

Imagine you're looking for one student's record.

Bad way

Read every record.

Record 1

Record 2

Record 3

...

Record 10,000,000

Very slow.

Good way

Use an index.

Think of an index like the index at the back of a textbook.

Instead of reading all 500 pages, you check:

Database → Page 327

You immediately find the answer.

Much faster.

What Went Wrong?

The database **should** have used the index.

Instead, it ignored it and scanned every row.

That's why it became slow.

Why Didn't It Use the Index?

Because SQL Server keeps statistics.

Statistics tell SQL Server things like:

- How many rows exist?
- How the data is distributed?
- Which route is fastest?

A recent database change made those statistics outdated.

So SQL Server made a bad decision.

Think of it like Google Maps using traffic information from last week.

It chooses a bad route.

Step 5: Confirm the Root Cause

Before changing anything, you test your theory.

You tell SQL Server:

"Use this index."

The query becomes much faster.

Now you know:

✅ The database choosing the wrong execution plan is the real problem.

Step 6: Fix It

You:

- Updated the database statistics.
- Optimized the stored procedure.
- Ensured the correct index would be used.

Step 7: Look for Other Improvements

While investigating, you notice another issue.

The API is sending too much data.

Example:

The screen shows:

Customer Name

Balance

Account Number

But the API sends:

Name

Balance

Account Number

Address

Phone

Email

Birth Date

Transaction History

Preferences

...

Most of this isn't even displayed.

That's unnecessary.

So you remove unused fields.

Another Improvement: Pagination

Suppose a customer has **10,000 investments**.

Instead of loading all 10,000 at once:

1

2

3

...

10000

You only load the first 50.

When the user clicks **Next**, you load the next 50.

This is called **pagination**.

It's much faster.

Final Result

Before:

- Response time: **10+ seconds**
- Customers complained.

After:

- Response time: **Under 2 seconds**
- Customers were happy.

Possible Sample Questions

1. Tell me about a time when you were trying to understand a complex problem on your team and you had to dig into the details to figure it out. Who did you talk with or where did you have to look to find the most valuable information? How did you use that information to help solve the problem?
2. Tell me about a situation that required you to dig deep to get to the root cause. How did you know you were focusing on the right things? What was the outcome? Would you have done anything differently?
3. Tell me about a problem you had to solve that required in-depth thought and analysis. How did you know you were focusing on the right things? What was the outcome? Would you have done anything differently?

Situation/Task

“One example that comes to mind was when I was at Accenture working on an investment platform. The system had been running fine in production, but one day we started getting repeated customer complaints about pages timing out when they tried to pull up their investment portfolio. This was critical because advisors and end customers depended on those dashboards for real-time decisions, and even a 5–10 second delay felt like a broken experience. My task was to figure out what was going wrong and fix it without disrupting live traffic. At first glance it looked like just another latency issue, but the tricky part was that it wasn’t consistent — some users reported timeouts, others were fine. That pushed me to go much deeper into the stack to understand where the real bottleneck was.”

Action

“The first thing I did was to dig through our logs and monitoring dashboards. At the API level, I noticed spikes in response times during peak traffic, but nothing obvious that pointed to one endpoint. So, I decided to trace a single user journey end-to-end: Angular front-end requests, our ASP.NET Web API layer, and the SQL Server backend. I set up detailed logging at each stage of the API call and ran a few load tests in our staging environment to reproduce the slowness. From those traces, I found that the actual delay was happening at the database level — some queries were taking hundreds of milliseconds

longer than expected. Next, I pulled execution plans from SQL Server and noticed that one of the main queries generating portfolio details was not using indexes properly. It was scanning millions of rows instead of leveraging the composite index we thought it was using. I dove deeper and realized that a recent schema change had altered column statistics, which meant the optimizer was picking the wrong plan. To validate, I worked with our DBA and ran the same query with query hints, and the performance improved drastically. That gave me confidence that the database layer was the root cause. I then rewrote the stored procedure to explicitly force the correct index usage and added missing statistics updates. I also suggested we add periodic jobs to refresh statistics automatically. At the same time, I looked at the API side. The Web API was serializing a large JSON payload with redundant fields that the UI didn't even render. I optimized the DTOs, trimmed unused fields, and introduced pagination for large portfolio lists so the front-end didn't try to render thousands of records at once."

Result

"After rolling out those changes, we saw response times drop from over 10 seconds to under 2 seconds on average, even during peak hours. Customer complaints dropped to almost zero, and the advisors specifically sent feedback that the dashboards were 'usable again' in real-time. From a business perspective, it helped us avoid what could have been a loss of trust with highvalue clients. Technically, the project taught me the importance of not stopping at the surface metrics but digging all the way through the stack — from database execution plans to API payload design — to get to the root cause. If I reflect on what I'd do differently, I'd say setting up more proactive monitoring and alerting earlier would have saved us some firefighting. Since then, I've been very deliberate about adding database query profiling and log correlation into every project I work on. Overall, this experience reinforced for me what it really means to dive deep: don't just patch the surface, keep peeling back until you've identified the single biggest bottleneck and fixed it in a way that scales."

Entity Framework Migration Commands

```
dotnet ef migrations list
dotnet ef migrations remove
```

Employee Model

The *Employee* model represents employee data in the application. It includes basic employee details, the related department ID, and a navigation property for the associated department.

```
namespace StudentManagementSystem.Models
{
    public class Employee
    {
        public Guid EmployeeId { get; set; }
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
        public Guid DepartmentId { get; set; }
        public Department? department { get; set; }
    }
}
```

Application Database Context

The *ApplicationDbContext* class connects the application to Entity Framework Core. It registers the employee and department tables so they can be queried and updated through the application.

```
using Microsoft.EntityFrameworkCore;
using StudentManagementSystem.Models;

namespace StudentManagementSystem.Data
{
    public class ApplicationDbContext : DbContext
    {
        public
ApplicationDbContext(DbContextOptions<ApplicationDbContext>
options) : base(options)
        {
```

```
        }

        public DbSet<Employee> employees { get; set; }
        public DbSet<Department> departments { get; set; }
    }
}
```

{

Application Settings

The application settings define logging behavior, allowed hosts, and the SQL Server connection string used by Entity Framework Core.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection":
"Server=.;Database=EmployeeDepartmentDB;Trusted_Connection=true;
TrustServerCertificate=true;"
  }
}
```

Program Configuration

The *Program.cs* file configures application services, enables controllers and Swagger, registers the database context, and sets up the HTTP request pipeline.

```
using Microsoft.EntityFrameworkCore;
using StudentManagementSystem.Data;

var builder = WebApplication.CreateBuilder(args);
```

```
// Add services to the container.
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddDbContext<ApplicationDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("
DefaultConnection"))
);

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Department DTOs

The department DTOs define the data shape used when creating or updating department records through the API.

```
namespace StudentManagementSystem.Models
{
    public class AddDepartmentDto
    {
        public Guid DepartmentId { get; set; }
        public required string DepartmentName { get; set; }
    }

    public class UpdateDepartmentDto
    {
```

```
        public Guid DepartmentId { get; set; }
        public required string DepartmentName { get; set; }
    }
}
```

Employee DTOs

The employee DTOs define the data required to create or update employee records, including employee identity, contact information, and department assignment.

```
namespace StudentManagementSystem.Models
{
    public class AddEmployeeDto
    {
        public Guid EmployeeId { get; set; }
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
        public Guid DepartmentId { get; set; }
    }

    public class UpdateEmployeeDto
    {
        public Guid EmployeeId { get; set; }
        public required string FirstName { get; set; }
        public required string LastName { get; set; }
        public string? Email { get; set; }
        public Guid DepartmentId { get; set; }
    }
}
```

Departments Controller

The *DepartmentsController* exposes API endpoints for creating, reading, updating, and deleting department records.

```
using Microsoft.AspNetCore.Mvc;
using StudentManagementSystem.Data;
using StudentManagementSystem.Models;

namespace StudentManagementSystem.Controllers
{
    [Route("api/[controller]")]
```

```
    [ApiController]
    public class DepartmentsController : ControllerBase
    {
        private readonly ApplicationDbContext _dbContext;

        public DepartmentsController(ApplicationDbContext
dbContext)
        {
            _dbContext = dbContext;
        }

        [HttpGet]
        public IActionResult GetAllDepartments()
        {
            var departments = _dbContext.departments.ToList();
            return Ok(departments);
        }

        [HttpPost]
        public IActionResult AddDepartment(AddDepartmentDto
addDepartmentDto)
        {
            var newDepartment = new Department
            {
                DepartmentName = addDepartmentDto.DepartmentName
            };

            _dbContext.departments.Add(newDepartment);
            _dbContext.SaveChanges();

            return Ok(newDepartment);
        }

        [HttpGet]
        [Route("{Id:Guid}")]
        public IActionResult GetDepartmentById(Guid Id)
        {
            var department = _dbContext.departments.Find(Id);

            if (department is null)
            {
```

```
                return NotFound();
            }

            return Ok(department);
        }

        [HttpPut]
        [Route("{Id:Guid}")]
        public IActionResult UpdateDepartmentById(Guid Id,
UpdateDepartmentDto updateDepartmentDto)
        {
            var department = _dbContext.departments.Find(Id);

            if (department is null)
            {
                return NotFound();
            }

            department.DepartmentName =
updateDepartmentDto.DepartmentName;
            _dbContext.departments.Update(department);
            _dbContext.SaveChanges();

            return Ok(department);
        }

        [HttpDelete]
        [Route("{Id:Guid}")]
        public IActionResult DeleteDepartmentById(Guid Id)
        {
            var department = _dbContext.departments.Find(Id);

            if (department is null)
            {
                return NotFound();
            }

            _dbContext.departments.Remove(department);
            _dbContext.SaveChanges();

            return Ok(department);
```

```
        }
    }
}
```

Employees Controller

The *EmployeesController* exposes API endpoints for creating, reading, updating, and deleting employee records.

```
using Microsoft.AspNetCore.Mvc;
using StudentManagementSystem.Data;
using StudentManagementSystem.Models;

namespace StudentManagementSystem.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeesController : ControllerBase
    {
        private readonly ApplicationDbContext _dbContext;

        public EmployeesController(ApplicationDbContext
dbContext)
        {
            _dbContext = dbContext;
        }

        [HttpGet]
        public IActionResult GetAllEmployees()
        {
            var employees = _dbContext.employees.ToList();
            return Ok(employees);
        }

        [HttpPost]
        public IActionResult AddEmployee(AddEmployeeDto
addEmployeeDto)
        {
            var newEmployee = new Employee
            {
```

```
                FirstName = addEmployeeDto.FirstName,
                LastName = addEmployeeDto.LastName,
                Email = addEmployeeDto.Email,
                DepartmentId = addEmployeeDto.DepartmentId
            };

            _dbContext.employees.Add(newEmployee);
            _dbContext.SaveChanges();

            return Ok(newEmployee);
        }

        [HttpGet]
        [Route("{Id:Guid}")]
        public IActionResult GetEmployeeById(Guid Id)
        {
            var employee = _dbContext.employees.Find(Id);

            if (employee is null)
            {
                return NotFound();
            }

            return Ok(employee);
        }

        [HttpPut]
        [Route("{Id:Guid}")]
        public IActionResult UpdateEmployeeById(Guid Id,
UpdateEmployeeDto updateEmployeeDto)
        {
            var employee = _dbContext.employees.Find(Id);

            if (employee is null)
            {
                return NotFound();
            }

            employee.FirstName = updateEmployeeDto.FirstName;
            employee.LastName = updateEmployeeDto.LastName;
            employee.Email = updateEmployeeDto.Email;
```

```
            employee.DepartmentId =
updateEmployeeDto.DepartmentId;

            _dbContext.employees.Update(employee);
            _dbContext.SaveChanges();

            return Ok(employee);
        }

        [HttpDelete]
        [Route("{Id:Guid}")]
        public IActionResult DeleteEmployeeById(Guid Id)
        {
            var employee = _dbContext.employees.Find(Id);

            if (employee is null)
            {
                return NotFound();
            }

            _dbContext.employees.Remove(employee);
            _dbContext.SaveChanges();

            return Ok(employee);
        }
    }
}
```